

Recipe-Controlled Decoder Audit for Structural Knowledge-Graph Completion

Anonymous PRICAI 2026 Submission

Abstract. We present a recipe-controlled decoder audit (RCDA) for structural transductive knowledge-graph completion (KGC). The audit asks a simple reporting question: before attributing gains to an encoder or training recipe, what changes when the decoder is swapped under the same recipe? Using ComplEx and DistMult as the primary controlled pair, with targeted RotatE/TransE spot-checks, we evaluate seven benchmarks. On five standard KGs, ComplEx-vs-DistMult differences are modest but consistent under our recipe (+0.005 to +0.012 MRR), whereas CompGCN-style encoder effects vary more by dataset. On small KGs, decoder effects become the main diagnostic: Kinship shows a stable ComplEx advantage of +0.143 MRR (6 seeds), while UMLS favours ComplEx by +0.022 MRR in a clean 6-seed server rerun but reverses in an earlier provenance variant. We therefore treat small-KG decoder choice as recipe- and provenance-sensitive rather than as a fixed dataset winner. We further show that decoder choice interacts with encoder depth on WN18RR, and that under our recipe $L=0$ ComplEx on YAGO3-10 reaches 0.6971 ± 0.0048 MRR at $d=128$. The result is a compact audit protocol: report matched decoder rows, log small-KG provenance, and sweep decoder $\times$ depth before making encoder-level claims.

1 Introduction

A structural KG-completion model has three independent levers: the *decoder* (ComplEx [19], DistMult [21], RotatE [15], . . .), the *encoder* (e.g., CompGCN [20] message passing), and the *training recipe*. Prior audits cover recipe choice [12] and CompGCN-style encoders [24]; the head-to-head DistMult-vs-ComplEx decoder axis under one recipe has not been audited at comparable breadth. We study structural transductive KGC on seven benchmarks, primarily with ComplEx/DistMult and CompGCN-style encoders, plus a small RotatE/TransE spot-check.

Headline: decoder choice is a reporting confound. Our goal is not to introduce a new KGC architecture or to rank decoders universally. Instead, we turn decoder choice into an explicit audit axis. On standard large KGs, the ComplEx-vs-DistMult gap is small but consistent under our recipe; on small KGs, the same swap can change the diagnostic conclusion; and on WN18RR, the best encoder depth depends on the decoder. The actionable recommendation is therefore narrow and practical: structural KGC papers should report a matched DistMult-vs-ComplEx row, and should avoid treating decoder choice as an implementation detail when making encoder or recipe claims.

Contributions. (C1) We propose RCDA, a recipe-controlled decoder-audit protocol for structural KGC. (C2) We provide a seven-benchmark ComplEx/DistMult diagnostic with targeted RotatE/TransE spot-checks, separating shared-grid evidence from small-KG decoder-only sensitivity. (C3) We show that Kinship has a large stable ComplEx advantage, while UMLS changes winner across provenance variants; edges/relation and symmetry are useful descriptors but not winner rules. (C4) We show decoder–encoder interaction on WN18RR: optimal L depends on the decoder. (C5) We provide a recipe-controlled structural reference grid, including 0.6971 ± 0.0048 MRR on YAGO3-10 ($d=128$) and 0.5379 on CoDEX-M.

2 Related Work

Prior audits. Recipe audits show that older KGE models can match newer methods under careful training [1, 2, 12, 16]. Encoder audits decompose R-GCN/CompGCN-style gains and find that graph-structure modelling is often less decisive than expected [24]. Our increment is orthogonal: we hold one recipe fixed, vary the decoder, and report when the decoder check changes the diagnostic conclusion.

The decoder axis: prior conceptual hypothesis, no direct measurement. The decoder axis has been studied at the level of single-dataset benchmark numbers but not as a controlled comparison. The closest prior is [11] (AAAI 2018), who proposed an L1 regulariser on ComplEx imaginary parts that pushes them to zero for empirically-symmetric relations, on the conceptual ground that “parameters in [ComplEx] are superfluous for relations that are either symmetric or antisymmetric.” They did not measure how a vanilla DistMult-vs-ComplEx choice behaves under matched structural recipes. We do (Sec. 4.4); we also report that low-edge small KGs are recipe-sensitive, so relation symmetry and edges/relation should be treated as audit descriptors rather than sufficient winner rules. [8] introduced Simple which interpolates between DistMult and ComplEx via two embedding heads. [18] (the JMLR-extended ComplEx paper) reports DistMult and ComplEx side-by-side on FB15k and WN18 but does not include the low-edges-per-relation benchmarks (UMLS, Kinship) where the decoder axis is most sensitive.

Decoder and method families. Shallow embedding models such as TransE [4], DistMult [21], ComplEx [19], RotatE [15], TuckER [3], and HAKE [23] score triples from endpoint embeddings. GNN-encoder extensions add message passing [5, 14, 20]; path-based reasoners such as NBFNet, A*Net, and RED-GNN encode query-specific structure [22, 25, 26]. The closest decoder-axis hypothesis is [11], which regularises ComplEx imaginary parts for empirically symmetric relations; however, it does not audit the vanilla decoder choice under matched recipes. We also revisit the YAGO3-10 ComplEx-N3 line [9]: under our recipe, $L=0$ ComplEx is already near saturation at $d \approx 128$, with only a small single-seed gain at $d=256$.

3 Approach: Recipe-Controlled Decoder Audit

The audit varies three levers under one fixed training recipe: (i) **primary decoder** $\in \{\text{ComplEx}, \text{DistMult}\}$, with targeted RotatE/TransE checks; (ii) **encoder depth**

$L \in \{0, 2, 3\}$ ($L=0$ = pure embedding lookup); and (iii) **recipe** ablations. We refer to the protocol as RCDA (**R**ecipe-**C**ontrolled **D**ecoder **A**udit).

Encoder. A standard CompGCN [20] layer with Hadamard composition aggregates relation-conditioned neighbour messages, with residuals and layer norm. We use $L \in \{2, 3\}$ unless scanning depth, hidden $d=200$ on FB/WN and $d \in \{64, 128\}$ on YAGO3-10.

Decoders. DistMult scores $\langle \mathbf{h}_h \odot \mathbf{r}, \mathbf{h}_t \rangle$. ComplEx splits embeddings into real/imaginary halves and can model antisymmetric relations. We compare the common-practice equal- d setting rather than a parameter-matched setting; this is intentional because the audit targets the reporting choices typically hidden in structural KGC papers.

Training recipe. We train with 1-vs-all CE over all entities, label smoothing $\varepsilon=0.3$, AdamW, MultiStepLR drops at 60% and 80%, dropout 0.2, batch 1024/2048, and weight decay 10^{-4} . Training runs for 300/200/150 epochs on WN/FB/YAGO-style scales.

4 Experiments

4.1 Setup

Datasets and evaluation. We evaluate five standard transductive benchmarks (FB15k-237 [17], WN18RR [6], YAGO3-10 [10], CoDEx-M, CoDEx-L [13]) plus UMLS and Kinship for the small-KG decoder spotlight. We report filtered MRR / Hits@ $\{1, 3, 10\}$ under the standard protocol [4] with average-rank tie-breaking. CoDEx-M/CoDEx-L use the canonical Safavi et al. splits; inverse triples are added only to training. Headline rows use 6 seeds where stated and otherwise 3 seeds; the exact seed lists are retained in the run JSONs/logs.

Recipe and reproducibility details. Defaults are $d=200$ for FB, WN, CoDEx, UMLS, and Kinship; $d \in \{64, 128\}$ for YAGO3-10; $L \in \{0, 2, 3\}$; batch 1024 or 2048 (CoDEx-L 4096); dropout 0.2; AdamW with lr 5×10^{-4} and weight decay 10^{-4} ; MultiStepLR at $0.6T/0.8T$ with factor 0.3; label smoothing $\varepsilon=0.3$; and 1-vs-all CE over all entities. Training uses AMP/FP16 on a single NVIDIA RTX 2080 Ti (11 GB); large YAGO/CoDEx runs and distance-decoder spot-checks are therefore server-side experiments. We retain per-run JSONs/logs, seed lists, configs, and code snapshots, and will release an anonymized audit bundle with the final version; all reported metrics use average-rank tie-breaking.

Baselines. Baselines use published numbers for classical, neural, and path-based KGC models [4, 9, 15, 19–21, 23, 26]. They are orientation points, not recipe-matched competitors.

Table 1. Orientation and selected controlled results under filtered evaluation with average-rank tie-breaking. Published baselines and our controlled runs use different recipes and implementations; the table is not a recipe-matched SOTA comparison.

Dataset	Method	MRR	Hits@1	Hits@3	Hits@10
FB15k-237	RotatE	0.338	24.1	37.5	53.3
	CompGCN	0.355	26.4	39.0	53.5
	KG-ICL-6L	0.355	27.0	38.9	52.3
	NBFNet	0.415	32.1	45.4	59.9
	RCDA ($L=3$, ComplEx)	0.420 ± 0.002	32.6 ± 0.1	46.2 ± 0.3	60.8 ± 0.3
WN18RR	RotatE	0.476	42.8	49.2	57.1
	CompGCN	0.479	44.3	49.4	54.6
	KG-ICL-6L	0.442	39.7	46.5	52.4
	NBFNet	0.551	49.7	57.3	66.6
	RCDA ($L=0$, ComplEx)	0.485 ± 0.002	45.6 ± 0.3	50.1 ± 0.1	53.5 ± 0.3
	RCDA ($L=2$, RotatE)	0.493 ± 0.002	45.8 ± 0.4	50.6 ± 0.2	56.2 ± 0.3
YAGO3-10	ComplEx (in NBFNet)	0.587	51.0	63.1	70.0
	ComplEx-N3 ($d=2000$)	≈ 0.58	—	—	—
	NBFNet	0.563	48.0	61.0	70.8
	RCDA ($L=0$, $d=128$, 3 seeds)	0.697 ± 0.005	63.9 ± 0.7	73.7 ± 0.3	79.3 ± 0.5
	RCDA + N3 ($L=0$, $d=128$)	0.695 ± 0.003	63.5 ± 0.3	72.8 ± 0.2	79.7 ± 0.3
CoDEX-M	ComplEx (Safavi 2020)	0.337	26.2	—	47.6
	RCDA ($L=0$, $d=200$)	0.538 ± 0.000	43.0 ± 0.1	—	74.5 ± 0.2
	RCDA + N3 ($L=0$, $d=200$)	0.547 ± 0.003	43.8 ± 0.4	—	75.7 ± 0.3
CoDEX-L	RCDA ($L=2$, $d=200$)	0.480 ± 0.007	38.5 ± 0.7	—	66.3 ± 0.6
	RCDA ($L=0$, $d=200$)	0.540 ± 0.003	44.1 ± 0.2	—	72.7 ± 0.5

4.2 Main Results

Five datasets, three verdicts. Table 1 is an orientation table for positioning RCDA, not a superiority claim across recipes. On **YAGO3-10**, pure ComplEx ($L=0, d=128$) reaches MRR 0.6971 ± 0.0048 in the new server rerun; adding CompGCN reduces MRR to 0.678 ± 0.009 . On **WN18RR**, RotatE reaches 0.493 ± 0.002 but still trails NBFNet’s 0.551 , keeping path reasoning as a separate lever. On **FB15k-237**, the encoder helps ($0.406 \rightarrow 0.420$ MRR), but the NBFNet gap is within recipe-comparison uncertainty. On **CoDEx-M/L**, pure ComplEx is the strongest of our structural configurations, while CompGCN hurts most on CoDEx-L.

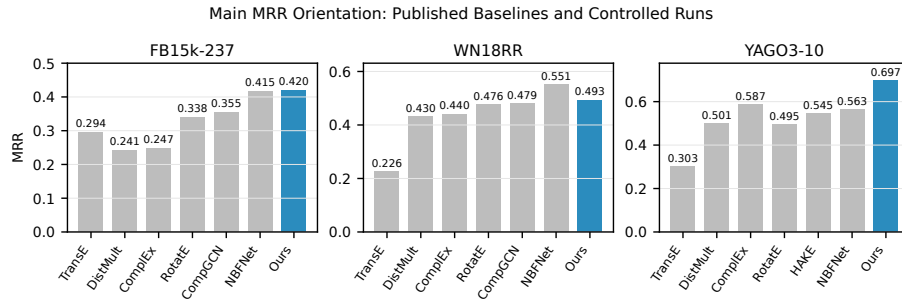


Fig. 1. Main MRR comparison on the standard transductive benchmarks. The figure is intended as orientation across published baselines and our controlled runs; WN18RR remains favourable to path-based reasoning.

4.3 The Decoder $\times$ Dataset Diagnostic

We now run the lever-by-lever audit. The central question is not whether decoders are globally more important than encoders, but whether hidden decoder choices can change the interpretation of structural KGC experiments.

Shared-grid and small-KG readings. On the five standard KGs where both axes are available, decoder Δ is modest ($+0.005$ to $+0.012$, spread 0.007), while the CompGCN-style encoder effect varies more ($+0.014$ to -0.061 , spread 0.075). This shared-grid result argues against a universal “decoder dominates encoder” claim. The decoder-only seven-dataset view is different: adding UMLS and Kinship expands the ComplEx-vs-DistMult range to 0.138 MRR, driven by Kinship and moderated by UMLS provenance sensitivity. The audit lesson is therefore conditional: decoder choice is a small but consistent reporting factor on standard KGs, and a potentially decisive confound on small KGs.

Table 2. Diagnostic axes under our fixed recipe (mean MRR; UMLS/Kinship 6 seeds, others 3; std < 0.01 on all cells). **Dec.** $\Delta = \text{MRR}_{\text{CX}} - \text{MRR}_{\text{DM}}$; **Enc.** $\Delta = \text{MRR}_{L \geq 2} - \text{MRR}_{L=0}$ on ComplEx. Encoder cells are available for the five standard KGs, while UMLS/Kinship are decoder-only small-KG diagnostics.

Dataset	CX MRR	DM MRR	Dec. Δ	Enc. Δ
UMLS	0.943	0.921	+0.022	—
Kinship	0.853	0.710	+0.143	—
FB15k-237	0.421	0.416	+0.005	+0.014
WN18RR	0.478	0.466	+0.012	−0.007
YAGO3-10	0.671	0.665	+0.006	−0.019
CoDEx-M	0.538	0.528	+0.010	−0.049
CoDEx-L	0.540	0.536	+0.005	−0.061
<i>Shared five-dataset spread:</i>			0.007	0.075
<i>Decoder-only seven-dataset spread:</i>			0.138	—

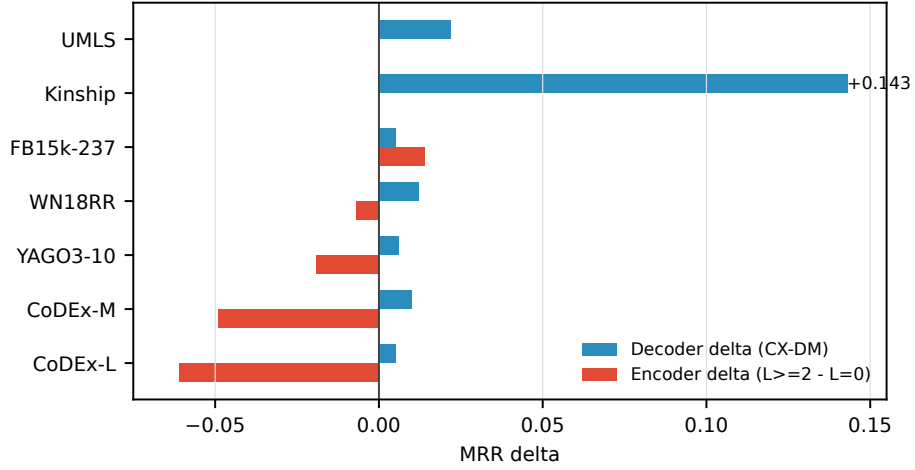


Fig. 2. Decoder Δ (ComplEx – DistMult) and encoder Δ ($L \geq 2$ vs. $L=0$ on ComplEx) by dataset. Standard-KG decoder gaps are small, while the decoder-only small-KG rows show why provenance-aware matched decoder checks matter.

The decoder Δ is small on standard KGs and large on small KGs. On the five standard transductive benchmarks (FB15k-237, WN18RR, YAGO3-10, CoDEx-M, CoDEx-L), ComplEx is ahead of DistMult by a modest +0.005 to +0.012 MRR (Table 2). On the two small biomedical / kinship benchmarks the gap is much larger, and UMLS changes winner across recipe variants. This motivates the small-KG diagnostic in Sec. 4.4.

4.4 Small-KG Sensitivity: UMLS and Kinship

A recipe-sensitive UMLS audit. A widely-used biomedical-KG tutorial states that “DistMult performs poorly on these datasets [UMLS, Kinship] as many relations are anti-symmetric in UMLS” [7]. Our new server rerun under the declared recipe gives UMLS ComplEx $\text{MRR} = 0.9427 \pm 0.0072$ vs. DistMult $= 0.9205 \pm 0.0051$ (6 seeds), so ComplEx is ahead by $+0.022$ MRR. However, an earlier run bundle gave DistMult $= 0.8705 \pm 0.0109$ vs. ComplEx $= 0.8344 \pm 0.0020$. We therefore treat UMLS as sensitivity evidence rather than a dataset-level decoder-winner claim. Its role is diagnostic: small-KG decoder conclusions can change across provenance/recipe variants that are often omitted from benchmark tables.

Symmetry is not enough. The natural first explanation is that UMLS is symmetric-rich, so DistMult’s enforced symmetry acts as a useful bias. Define $\text{sym}(r) = |\{(h, t) : (h, r, t), (t, r, h) \in \mathcal{D}_{\text{train}}\}| / |\{(h, t) : (h, r, t) \in \mathcal{D}_{\text{train}}\}|$, edge-weighted across relations: UMLS scores 0.133, Kinship 0.207 — UMLS is in fact less symmetric than Kinship. Dataset-level symmetry therefore cannot explain either the earlier UMLS reversal or the larger Kinship ComplEx advantage by itself.

Label smoothing alone is not the sensitivity source. In the earlier lock-in pass, we repeated UMLS at $\varepsilon \in \{0, 0.1, 0.3\}$ (3 seeds each, $d=200$, $L=2$): DistMult won at every ε , with decoder Δ MRR $= -0.0175$ ($\varepsilon=0$), -0.0268 ($\varepsilon=0.1$), -0.0446 ($\varepsilon=0.3$). This rules out label smoothing as the sole cause of that pass, but the clean server rerun shows the reversal is not robust enough for a headline claim.

First-cut descriptor: sample size per relation. ComplEx doubles DistMult’s relation parameter budget via real/imaginary parts, but extra capacity alone is not the whole story. In an earlier lock-in $d \in \{100, 200, 400\}$ scan, ComplEx is monotone in d on UMLS while DistMult has a $d=200$ sweet spot; the clean rerun nevertheless changes the UMLS winner. The most useful descriptor we found is training edges per relation: UMLS has only 113 e/r and Kinship 342, far below FB15k-237 (1148), CoDEX-M (3627), WN18RR (7894), and YAGO3-10 (29 163). Synthetic per-relation subsamples of FB15k-237 and WN18RR reproduce a low-e/r DistMult preference in sign, but absolute MRR is low under aggressive subsampling, so e/r should be read as a hypothesis-generating audit descriptor rather than a mechanism proof or winner rule.

4.5 YAGO3-10: Saturation Under Our Recipe at $d \approx 128$

Setup and scope. [9] reported ≈ 0.58 MRR on YAGO3-10 with ComplEx-N3 at $d=2000$ (Adagrad, sampled-negatives logistic loss). The original paper introduced N3 regularisation; it did not explicitly assert that capacity *is* the bottleneck. We do not argue against the Lacroix-2018 result itself. Our dimension scan ($L=0$ ComplEx, our recipe held fixed; 3 seeds through $d=128$ plus a single-seed $d=256$ spot-check) shows recipe-conditional near-saturation well below $d=2000$:

Table 3. Edges per relation as a first-cut audit descriptor. CoDEx-L is excluded from this predictor analysis although its $L=0$ decoder spot-check is reported in Table 2.

Dataset	#train	#rel	e/r	Dec. Δ
UMLS	5,216	46	113	+0.022
Kinship	8,544	25	342	+0.143
FB15k-237	272,115	237	1,148	+0.005
CoDEx-M	185,000	51	3,627	+0.010
WN18RR	86,835	11	7,894	+0.012
YAGO3-10	1,079,040	37	29,163	+0.006

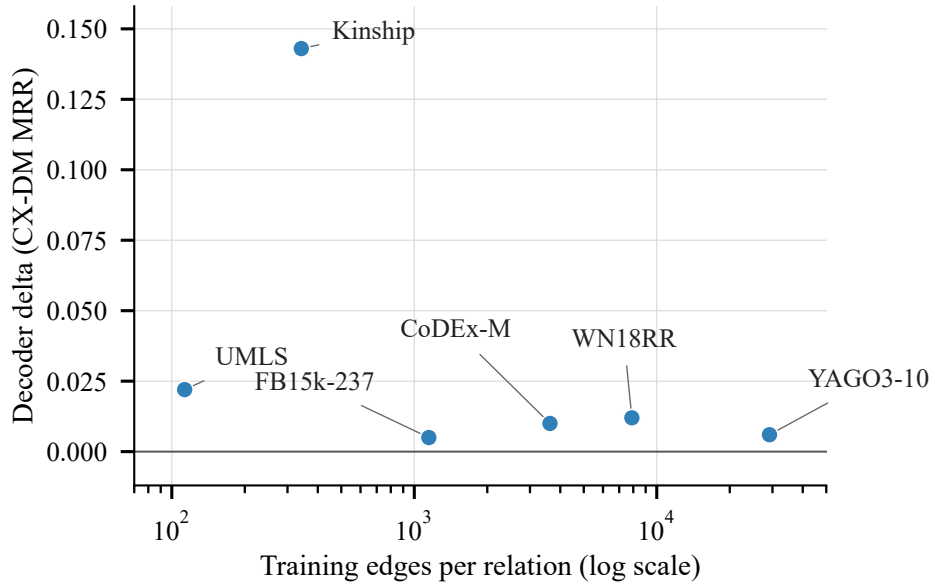


Fig. 3. Decoder preference as a function of training edges per relation. Low-e/r datasets produce the largest decoder gaps, but UMLS shows that e/r is an audit descriptor rather than a winner rule.

Table 4. Targeted distance-decoder spot-checks ($d=200$, $L=2$). RotatE/TransE use memory-safe batch/chunk settings on the RTX 2080 Ti, so the table is diagnostic and not recipe-matched to the primary DistMult/ComplEx pair.

Dataset	RotatE	TransE
UMLS	0.8490 ± 0.0065	0.1005 ± 0.0087
Kinship	0.7266 ± 0.0091	0.0472 ± 0.0044
WN18RR	0.4929 ± 0.0019	0.2016 ± 0.0045

Table 5. YAGO3-10 capacity scan. $L=0$ ComplEx. MRR is near saturation by $d=128$; a new $d=256$ spot-check gives $+0.008$ at $2\times$ params. The new $d=128$ server rerun gives 0.6971 ± 0.0048 .

d	64	100	128	256
MRR	0.671	0.691	0.697	0.705
Δ vs prev	—	+0.019	+0.006	+0.008
Params (M)	7.9	12.3	15.8	31.6

Reading. Under our recipe with the ComplEx decoder, the $d=64 \rightarrow 100$ step gives $+0.019$ MRR, while later increases are smaller: $+0.006$ at $d=128$ and $+0.008$ in a single-seed $d=256$ spot-check. Thus $d=128$ is already near saturation for this configuration, although not a hard optimum. We do not claim this saturation transfers to all recipes, optimisers, or decoders; nor that the Lacroix-2018 number is reproducible *at* $d=128$ under their original recipe (we did not re-run their setup at our d , but doing so would directly test the recipe-vs-capacity confound). Adding N3 to our recipe at $d=128$ changes YAGO3-10 MRR by -0.002 , suggesting N3 and our recipe are largely orthogonal *at this* d ; we make no claim about N3 at $d=2000$ or under other recipes.

Where the YAGO3-10 gain comes from. A per-relation audit shows that the aggregate is driven by frequent dense relations rather than by a few tiny relations: *playsFor* and *isAffiliatedTo* alone cover roughly 60% of the YAGO3-10 test triples and both reach high MRR under the pure ComplEx recipe. Low-MRR relations such as *edited*, *actedIn*, and *influences* are long-tail or require textual/contextual cues, so the result should be read as a structural recipe fit to dense sub-schema rather than a universal KG-completion solution.

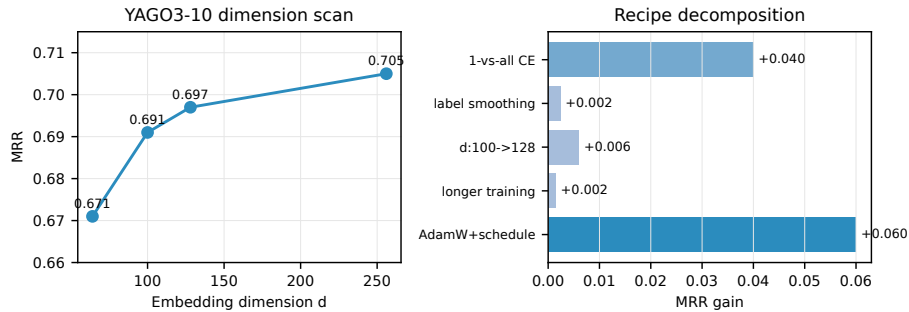


Fig. 4. YAGO3-10 spotlight: dimension saturation and leave-one-out recipe diagnostics. The difference from older ComplEx settings is associated mainly with the loss and optimiser/learning-rate schedule in this implementation, not simply with increasing d .

Table 6. Recipe diagnostics on YAGO3-10. Leave-one-out from our selected configuration; “original” denotes the Trouillon-2016 ComplEx setting.

Recipe component (original $\rightarrow$ ours)	Δ MRR
Logistic loss + 50 sampled neg. $\rightarrow$ 1-vs-all CE	+0.040
$\varepsilon = 0 \rightarrow \varepsilon = 0.3$	+0.0024
$d = 100 \rightarrow d = 128$	+0.006
50 epochs $\rightarrow$ 150 epochs	+0.0015
Adagrad $\rightarrow$ AdamW + MultiStepLR	+0.060
Total	+0.110

4.6 Encoder Depth $\times$ Decoder Choice Interact on WN18RR

The interaction. Most CompGCN-lineage work fixes the decoder (typically ComplEx or DistMult), sweeps the encoder depth L , and reports the optimal depth as if it were a property of the dataset. We measure this assumption on WN18RR ($d=200$, recipe fixed, 3 seeds per cell):

Table 7. WN18RR encoder-depth $\times$ decoder-choice ($d=200$, 3 seeds per cell, mean MRR). Decoders peak at opposite ends ($L=0$ ComplEx, $L=3$ DistMult) and share an $L=1$ dip. For DistMult, $L=0$ ties the $L=3$ peak within seed std.

Decoder $\backslash L$	$L=0$	$L=1$	$L=2$	$L=3$
ComplEx	0.485	0.453	0.481	0.478
DistMult	0.473	0.441	0.467	0.474

Peaks at opposite ends; shared $L=1$ dip. Three robust patterns appear: ComplEx peaks at $L=0$ (0.4849), DistMult at $L=3$ (0.4743); both decoders share an $L=1$ dip; and DistMult’s $L=3$ peak is only +0.0012 MRR above $L=0$, within seed std. A fixed-decoder ablation would conclude “encoder hurts” or “encoder neutral”, while a fixed-encoder ablation would conclude “ComplEx $>$ DistMult”. Neither captures the 4×2 interaction.

Implication. Encoder utility is dataset- and decoder-shaped, not uniformly low. On FB15k-237, the encoder helps ComplEx by +0.014 MRR; on WN18RR, it is neutral or harmful for ComplEx. Small KGs differ again: $L=0$ collapses on UMLS/Kinship, so the WN18RR observation does not generalise to all sparse settings.

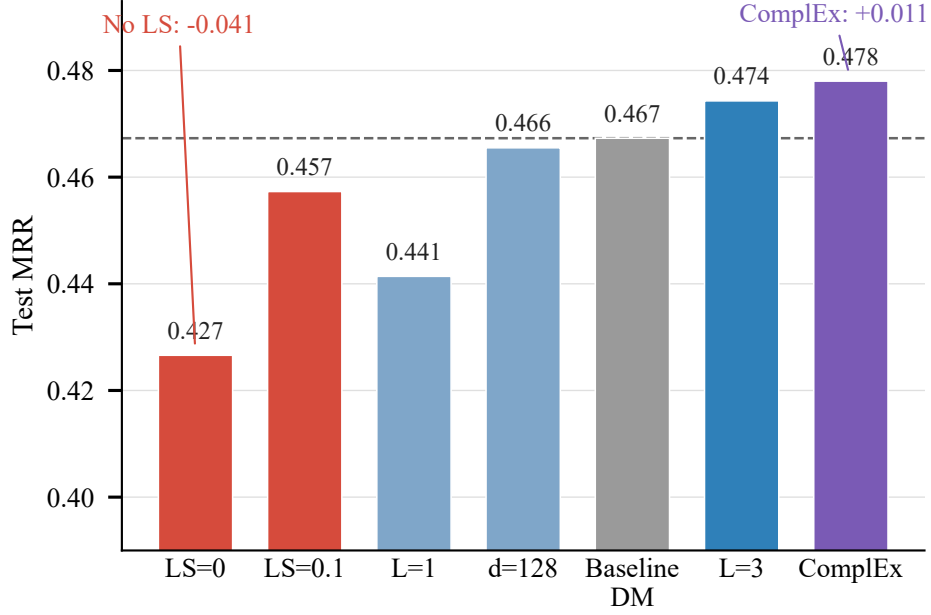
4.7 Training Recipe Ablation

To identify which training choices drive our results, we ablate six ingredients of the recipe on WN18RR (3 seeds) and FB15k-237 (single seed, 3-seed for headline rows), starting from the main configuration for each dataset.

Table 8. Training-recipe ablation on WN18RR ($d=200$, $L=2$, 3 seeds; mean MRR with $H@k$ in %). Label smoothing is the most impactful single ingredient.

Configuration	MRR	H@1	H@3	H@10
Baseline ($d=200$, $L=2$, $LS=0.3$, DM)	0.467	43.6	48.0	52.5
+ deeper GCN ($L=3$)	0.474	44.3	48.9	53.1
+ ComplEx decoder	0.478	45.1	49.1	52.8
– smaller dim ($d=128$)	0.466	43.6	47.8	52.0
– shallower ($L=1$)	0.441	41.7	45.1	48.7
– weaker smoothing ($LS=0.1$)	0.457	43.0	46.7	50.6
– no smoothing ($LS=0.0$)	0.427	40.7	43.4	46.1

Label smoothing is dominant. Removing label smoothing ($\varepsilon = 0$) drops MRR by -0.041 on WN18RR (Table 8) and -0.017 on FB15k-237; reducing to $\varepsilon = 0.1$ already costs $-0.010/-0.020$. This is larger than any architectural change we tested.

**Fig. 5.** WN18RR recipe ablation. Label smoothing is the largest single ingredient in this recipe, while depth and decoder changes are smaller and interact with each other.

Shallow GCN, decoder picks up the slack. A single-layer GCN loses -0.029 MRR on WN18RR; $L=2 \rightarrow L=3$ gives $+0.004$ on WN18RR, $+0.010$ on FB15k-237, but reverses sign on YAGO3-10 (-0.022). Replacing DistMult with ComplEx yields $+0.011$

on WN18RR, +0.003 on FB15k-237, and +0.006–0.011 across L on YAGO3-10. The cross-dataset decoder pattern generalises to all five transductive benchmarks (Sec. 4.3).

4.8 YAGO3-10 Recipe Diagnostics plus Sensitivity / Efficiency

Our $L=0$ ComplEx on YAGO3-10 (0.6971) is +0.110 MRR relative to the Trouillon-2016 ComplEx number reported in the NBFNet comparison table [26]. A 3-seed leave-one-out diagnostic associates the difference with the loss change (+0.040), smoothing/training length/ d (+0.010), and optimiser/initialisation residual (+0.060). We read this as implementation-level sensitivity rather than a causal proof that any single ingredient dominates. N3 is largely orthogonal at saturated d . The resulting configuration is also computationally light after graph encoding: on one RTX 2080 Ti, $L=0$ ComplEx scores about 112K YAGO3-10 queries/s at $d=128$, while $L=2$ DistMult at $d=64$ scores about 297K queries/s after a 25.9 ms graph-encoding pass.

5 Discussion

The WN18RR gap to NBFNet. We do *not* claim superiority over path-based models: on WN18RR, NBFNet’s 0.551 stays 5.8 MRR above our RotatE spot-check (0.493) and 6.6 above our selected ComplEx configuration (0.485). The gap is consistent with path propagation having a genuine advantage on tree-like hypernym chains. Path-based reasoning is a separate lever, orthogonal to our decoder-vs-CompGCN-encoder audit.

6 Conclusion

Prior controlled audits cover recipe [12] and CompGCN-style encoder [24] axes; this paper adds the decoder axis through RCDA. The actionable outcome is a compact audit checklist: report a matched DistMult-vs-ComplEx row, log small-KG recipe and provenance details, inspect edges per relation and symmetry as descriptors, and avoid reporting encoder depth without the decoder dimension. On standard KGs, decoder gaps are modest but consistent under our recipe; on small KGs, they can become decisive and provenance-sensitive. Future encoder ablations should report the full $\{\text{decoder}\} \times \{L\}$ grid rather than a row-wise minimum.

Limitations

Our conclusions are limited to structural transductive KGC with the ComplEx/DistMult decoder family, CompGCN-style encoders, and one fixed recipe; they do not extend to inductive, text-augmented, LLM-based, OGB-scale, or path-based KGC. A fixed recipe isolates decoder and encoder effects but does not identify each model’s independently tuned optimum, and recipe itself is a large lever. Table 1 mixes published baselines and our recipe-controlled runs, so small gaps should be read cautiously. The decoder and encoder Δ columns are not fully symmetric: UMLS/Kinship are decoder-only small-KG diagnostics, seed budgets and depths differ, and YAGO3-10 dimensionality differs across

cells. The UMLS mechanism remains a heuristic: edge-weighted symmetry is coarse, UMLS/Kinship are small benchmarks, e/r subsampling can reduce absolute MRR, and UMLS changes winner between an earlier run bundle and the clean server rerun. Finally, throughput is scoring throughput after graph encoding, not full end-to-end training or inference wall-clock.

Ethics Statement

Improved KG completion benefits downstream applications (search, recommendation, biomedical KG construction) but can amplify biases in the source KG. Our recipe reduces GPU-hours per trained model and brings reproduction within reach of single-11-GB-GPU groups. We encourage per-relation error analysis and fairness auditing before deployment. All datasets used are public.

Bibliography

- [1] Akrami, F., Saeef, M.S., Zhang, Q., Hu, W., Li, C.: Realistic re-evaluation of knowledge graph completion methods: An experimental study. In: Proceedings of the ACM SIGMOD International Conference on Management of Data (2020)
- [2] Ali, M., Berrendorf, M., Hoyt, C.T., Vermue, L., Galkin, M., Sharifzadeh, S., Fischer, A., Tresp, V., Lehmann, J.: Bringing light into the dark: A large-scale evaluation of knowledge graph embedding models under a unified framework. In: IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI) (2021)
- [3] Balažević, I., Allen, C., Hospedales, T.M.: TuckER: Tensor factorization for knowledge graph completion. In: Conference on Empirical Methods in Natural Language Processing (EMNLP) (2019)
- [4] Bordes, A., Usunier, N., Garcia-Durán, A., Weston, J., Yakhnenko, O.: Translating embeddings for modeling multi-relational data. In: Advances in Neural Information Processing Systems (2013)
- [5] Chen, S., Liu, X., Gao, J., Jiao, J., Zhang, R., Ji, Y.: HittER: Hierarchical transformers for knowledge graph embeddings. In: Conference on Empirical Methods in Natural Language Processing (EMNLP) (2021)
- [6] Dettmers, T., Minervini, P., Stenetorp, P., Riedel, S.: Convolutional 2d knowledge graph embeddings. In: AAAI Conference on Artificial Intelligence (2018)
- [7] Graph4NLP Authors: Knowledge graph completion — Graph4NLP v0.4.1 documentation. <https://graph4ai.github.io/graph4nlp/guide/classification/kgcompletion.html> (2021), accessed 2026-05-04
- [8] Kazemi, S.M., Poole, D.: SimpleE embedding for link prediction in knowledge graphs. In: Advances in Neural Information Processing Systems (2018)
- [9] Lacroix, T., Usunier, N., Obozinski, G.: Canonical tensor decomposition for knowledge base completion. In: International Conference on Machine Learning (ICML). pp. 2863–2872 (2018)
- [10] Mahdisoltani, F., Biega, J., Suchanek, F.M.: YAGO3: A knowledge base from multilingual Wikipedias. In: Conference on Innovative Data Systems Research (CIDR) (2015)
- [11] Manabe, H., Hayashi, K., Shimbo, M.: Data-dependent learning of symmetric/antisymmetric relations for knowledge base completion. In: AAAI Conference on Artificial Intelligence (2018)
- [12] Ruffinelli, D., Broscheit, S., Gemulla, R.: You CAN teach an old dog new tricks! on training knowledge graph embeddings. In: International Conference on Learning Representations (ICLR) (2020)
- [13] Safavi, T., Koutra, D.: CoDEx: A comprehensive knowledge graph completion benchmark. In: Conference on Empirical Methods in Natural Language Processing (EMNLP) (2020)
- [14] Schlichtkrull, M., Kipf, T.N., Bloem, P., van den Berg, R., Titov, I., Welling, M.: Modeling relational data with graph convolutional networks. In: European Semantic Web Conference (2018)

- [15] Sun, Z., Deng, Z.H., Nie, J.Y., Tang, J.: RotatE: Knowledge graph embedding by relational rotation in complex space. In: International Conference on Learning Representations (2019)
- [16] Sun, Z., Vashishth, S., Sanyal, S., Talukdar, P., Yang, Y.: A re-evaluation of knowledge graph completion methods. In: Annual Meeting of the Association for Computational Linguistics (ACL) (2020)
- [17] Toutanova, K., Chen, D., Pantel, P., Poon, H., Choudhury, P., Gamon, M.: Representing text for joint embedding of text and knowledge bases. In: Conference on Empirical Methods in Natural Language Processing (2015)
- [18] Trouillon, T., Dance, C.R., Welbl, J., Riedel, S., Gaussier, É., Bouchard, G.: Knowledge graph completion via complex tensor factorization. *Journal of Machine Learning Research* **18**, 1–38 (2017)
- [19] Trouillon, T., Welbl, J., Riedel, S., Gaussier, É., Bouchard, G.: Complex embeddings for simple link prediction. In: International Conference on Machine Learning (2016)
- [20] Vashishth, S., Sanyal, S., Nitin, V., Talukdar, P.: Composition-based multi-relational graph convolutional networks. In: International Conference on Learning Representations (2020)
- [21] Yang, B., Yih, W.t., He, X., Gao, J., Deng, L.: Embedding entities and relations for learning and inference in knowledge bases. In: International Conference on Learning Representations (2015)
- [22] Zhang, Y., Yao, Q.: Knowledge graph reasoning with relational digraph. In: The Web Conference (2022)
- [23] Zhang, Z., Cai, J., Zhang, Y., Wang, J.: Learning hierarchy-aware knowledge graph embeddings for link prediction. In: AAAI Conference on Artificial Intelligence (2020)
- [24] Zhang, Z., Wang, J., Ye, J., Wu, F.: Rethinking graph convolutional networks in knowledge graph completion. In: Proceedings of the ACM Web Conference (WWW) (2022)
- [25] Zhu, Z., Yuan, X., Galkin, M., Xhonneux, S., Zhang, M., Gazeau, M., Tang, J.: A*Net: A scalable path-based reasoning approach for knowledge graphs. In: Advances in Neural Information Processing Systems (2023)
- [26] Zhu, Z., Zhang, Z., Xhonneux, L.P., Tang, J.: Neural Bellman-Ford networks: A general graph neural network framework for link prediction. In: Advances in Neural Information Processing Systems (2021)